

PDF 1 of 3 — Original English Source

Scalability

The complete, unedited lesson exactly as published — every paragraph, heading, table, callout and example preserved. This document is the source of record for the topic.

100% Original Content

Nothing Summarized

Nothing Removed

Scalability is the ability of a system to handle an increasing workload, either by adding more resources or by making the existing ones bigger.

Those are the two directions, and they have names.

Horizontal Scaling

Horizontal scaling, also called scaling out, means adding more machines or nodes to a system so the workload is distributed evenly across them.

No single node has to absorb the growth. More requests arrive, more machines share them.

This suits distributed systems well. Capacity can be added while the system runs, which makes it a cost-effective way to handle workloads that rise and fall, and having many machines is what makes high availability possible in the first place.

Cassandra and MongoDB are good examples. Both are built so you add machines to meet growing demand.

Vertical Scaling

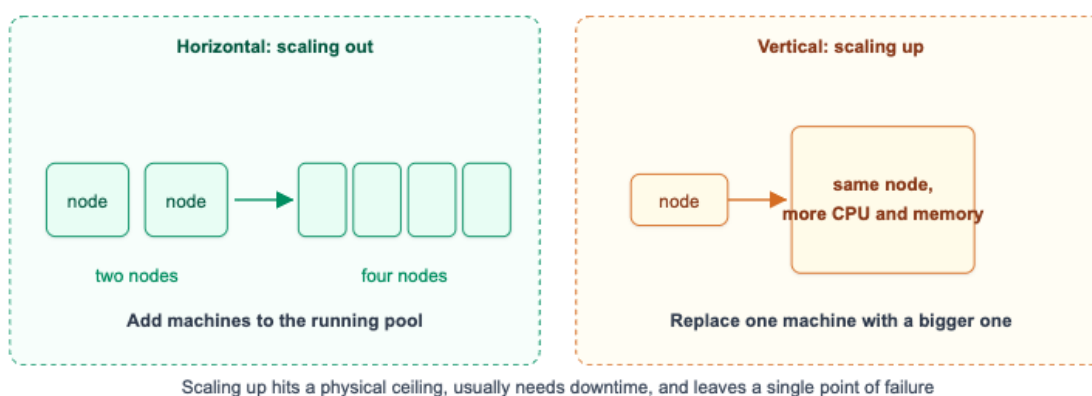
Vertical scaling, also called scaling up, means increasing the capacity of an individual node by upgrading its hardware: more CPU, more memory, more storage.

The same single machine now handles more work.

MySQL is a good example. Moving from a smaller machine to a bigger one is a straightforward way to get more capacity, and the process usually involves downtime.

The same single machine now handles more work.

MySQL is a good example. Moving from a smaller machine to a bigger one is a straightforward way to get more capacity, and the process usually involves downtime.



Scaling out adds more machines beside the existing ones, while scaling up replaces one machine with a bigger one

The Difference That Decides It

Horizontal scaling is easier to do dynamically. You add machines to the pool that is already running.

Vertical scaling is bounded by a single server's capacity. Growing past that ceiling usually means downtime, and there is an upper limit you eventually hit no matter what you spend.

	Horizontal (scaling out)	Vertical (scaling up)
What changes	Number of machines	Size of one machine
Ceiling	Practically far away	The biggest machine available
Adding capacity	Add nodes to the running pool	Usually needs downtime
Failure of one node	The others carry on	Everything is on that node
Examples given	Cassandra, MongoDB	MySQL

Read the last two rows together, because that is where the risk lives. Relying only on vertical scaling has two problems at once: there is a physical ceiling on how big one machine can get, and putting everything on that one machine creates a single point of failure. The system gets faster right up until the moment it is completely down.

Interview Tip

In an interview, do not just say "I would scale horizontally." Say what makes it possible. Horizontal scaling only works if requests can go to any node, which means saying where the state lives. "The service is stateless, so I can put any number of instances behind a load balancer, and session state goes in a shared cache" is the actual design.

Key takeaway

Scalability is handling more work by adding resources. Horizontal scaling adds machines and spreads the workload across them, which is the approach distributed systems are built for. Vertical scaling makes one machine bigger, which is simple but limited by that machine's ceiling, usually requires downtime, and concentrates everything into a single point of failure.

The next lesson, Availability, covers what it takes to keep a system reachable once it is spread across many machines.